Ce valoare are x în codul din dreapta?

- 3
- 4
- 12
- eroare ✓

Correct

Response saved

```
x = let x = 3, y = 4 in x * y
```

Ce valoare au x și y în codul din dreapta?

- x = 2 și y = 2
- x = 2 și y = 1
- x = 1 și y = 1 ✓
- eroare

Correct

Response saved

```
x = 1
y = let y = 2 in x
```

Ce valoare are f 5 în codul din dreapta?

- 6
- 2 ✓
- 1
- eroare

Correct

Response saved

```
f x = g y
  where
    g x = x + 1
    y = 1
```

Ce tip are expresia (42 :: Int,True)?

[Int,Bool]

(Int,Bool)

Tuple

eroare

Incorrect

Response saved

De ce expresia 3 * "FOO" nu este valida in Haskell?

- Pentru ca "FOO" nu este o valoare numerica, ci un sir de caractere. ✓
- Pentru ca operatorul * este predefinit doar pentru tipul Int.
- Pentru ca "FOO" ar trebui sa fie scris ca 'FOO'.
- Pentru ca in Haskell nu putem folosi operatori pe siruri de caractere.

Correct

Response saved

Care este signatura unei functii care primeste ca argumente
- un Bool si
- o functie de tip Int -> Int,
si intoarce o functie de tip Char -> String?

f :: Bool -> (Int -> Int) -> (Char -> String)

f :: (Bool, Int -> Int) -> Char -> String

f :: Bool -> Int -> Int -> Char -> String

f :: (Bool -> Int) -> Int -> (Char, String)

Correct

Response saved

Care din expresiile de mai jos nu defineste lista [3,4,5,6]?

- 3 : 4 : 5 : 6 ✓
- 3 : 4 : 5 : 6 : []
- [3 .. 6]
- 3 : 4 : 5 : [6]

Correct

Response saved

Ce obtinem dupa instructiunile?

nu se poate aplica functia zip

[(1,11),(2,12)] ✓

[1,2,3,11,12]

[(1,11),(1,12),(2,11),(2,12),(3,11),(3,12)]

Correct

Response saved

```
let xs = [1,2,3]
let ys = [11,12]
zip xs ys
```

Ce optinem dupa instructiunile?

6

5

[0,1,2,3,4]

4

Incorrect

Response saved

```
let naturals = [0..]
natural !! 5
```

Ce tip are functia curry foo?

- nu se poate aplica functia curry peste foo ✓
- Int -> Char -> String -> String ✗
- Int -> (Char -> String) -> String
- (Int -> Char -> String) -> String

Incorrect

Response saved

```
foo :: (Int,Char,String) → String
```

Ce valoare are expresia g . f $ 3?

6

18

eroare

36

Correct

Response saved

```
f x = x + x
g x = x * x
```

Ce se obtine?

[10,9,8]

[1,2,3]

eroare

[3,2,1]

Incorrect

Response saved

```
reverse . take 3 [1 .. 10]
```

Ce se obtine?

eroare

[2,3,4,5]

[0,1,2,3]

[0,-1,-2,-3]

Correct

Response saved

```
map (1-) [1,2,3,4]
```

Ce se obtine?

eroare

"huhu"

3

"uuu"

Correct

Response saved

```
length.filter (=='u') $ "buhuhu"
```

Ce se obtine?

- [2,4,6,8,10] ✓
- [1,3,5,7,9]
- eroare
- 5

Correct

Response saved

```
filter (\x → mod x 2 == 0) [1..10]
```

Ce se obtine?

"wootWOOTwoot"

eroare

["woot","WOOT","woot"]

3

Correct

Response saved

```
foldr (++) [“woot”,”WOOT”,”woot”]
```

Ce se obtine?

eroare

False

True

[True, False, True]

Correct

Response saved

```
foldr (&&) True [False,True]
```

Ce se obtine?

eroare

"(1+(2+(3+(4+(5+0)))))"

"1+2+3+4+5+0"

["(","1","2","3","4","5"]

Incorrect

Response saved

```
foldr (\x y → concat [“(”,x,”+”,y,”)”])
      “0” [“1”,”2”,”3”,”4”,”5”]
```

Ce se obtine?

[]

eroare

[1,2,3] ✓

[3,2,1]

Correct

Response saved

```
foldr (:) [] [1..3]
```

Ce este Doggies?

- constructor de tip ✓
- constructor de date
- tip de date produs
- niciunul din raspunsurile de mai sus

Correct

Response saved

```
data Doggies a =
      Husky a
    | Mastiff a
```

Ce tip are Mastiff "Scooby Doo"?

Doggies

[Char]

Doggies [Char]

Doggies Mastiff

Incorrect

Response saved

```
data Doggies a =
        Husky a
      | Mastiff a
```

Ce tip are Husky (10 :: Integer)?

- Doggies
- Doggies Integer ✓
- Integer
- eroare

Correct

Response saved

```
data Doggies a =
      Husky a
    | Mastiff a
```

Ce tip are f?

- Bool -> Bool -> Either Bool String ✗
- String -> String -> Either String String
- Bool -> Bool -> Either String Bool ✓
- Bool -> Bool -> Maybe String

Incorrect

Response saved

```
f x y | x && y = Right x
      | otherwise = Left “foo”
```

Care din urmatoarele functii poate avea tipul Bool -> Int -> [Bool]?

f x y = [0, y]

f x y = [x, True]

f x y = [y, True]

niciunul din raspunsurile de mai sus

Correct

Response saved

Ce tip are justBoth?

- a -> b -> [Maybe a, Maybe b] ✗
- a -> a -> [Just a]
- a -> b -> [Maybe a]
- a -> a -> [Maybe a] ✓

Incorrect

Response saved

```
justBoth a b =
    [Just a, Just b]
```

Clasa EQ

include toate tipurile din Haskell

coincide cu clasa Ord

face testarea egalității posibilă

nu este definită

Correct

Response saved

Ce tip are operatorul (>) din clasa Ord?

- Ord a => a -> a -> Bool ✓
- Ord a => Int -> Bool
- Ord a => a -> Char
- Ord a => Char -> [Char]

Correct

Response saved

Ce puteți spune despre codul din dreapta?

- codul este corect
- codul nu este corect deoarece nu există o instanță a clasei Num pentru tipul Mood
- codul nu este corect deoarece nu există o instanță a clasei Ord pentru tipul Mood
- codul nu este corect deoarece nu există o instanță a clasei Eq pentru tipul Mood ✓

Correct

Response saved

```
data Mood = Blah
          | Woot
          deriving Show

settleDown x = if x == Woot
                  then Blah
                  else x
```

Care este signatura funcției fmap din clasa Functor f?

fmap :: (a -> b) -> f a -> f b

fmap :: (a -> b) -> f a

fmap :: ((a -> b) -> f a) -> f b

fmap :: (a -> b) -> f a -> f a

Correct

Response saved

Știm că const :: a -> b -> a.
Fie replaceWithP = const 'p'.
Ce întoarce replaceWithP (Just 10)?

Just 10

'p' ✓

Just 'p'

instrucțiune invalidă ✗

Incorrect

Response saved

Știm că const :: a -> b -> a.
Fie replaceWithP = const 'p'.
Ce întoarce fmap replaceWithP (Just 10)?

Just 10

'p'

Just 'p' ✓

instrucțiune invalidă ✗

Incorrect

Response saved

Ce întoarce expresia din dreapta?

instrucțiune incorecta

Right 1

Nothing 1

Just 1

Correct

Response saved

```
pure 1 :: Maybe Int
```

Ce întoarce expresia din dreapta?

instrucțiune incorecta

Right 1

Nothing 1

Just 1

Correct

Response saved

```
pure 1 :: Maybe Int
```

Ce tip are expresia din dreapta?

instrucțiune incorecta

Maybe [Char]

String

[Char]

Correct

Response saved

```
Just (++ “Hello ”) <*> Just “world!”
```

Ce întoarce expresia din dreapta?

- instrucțiune incorecta
- Just "Hello world!"
- "Hello world!"
- Just "world!Hello " ✓

Correct

Response saved

```
Just (++ “Hello ”) <*> Just “world!”
```

Care expresie este echivalentă cu blocul do din dreapta?

z >> \y -> s y >> return (f y)

z >>= \y -> s y >> return (f y) ✓

z >> \y -> s y >>= return (f y)

nicio variantă

Correct

Response saved

```
do y ← z
   s y
   return (f y)
```

Ce tip are expresia din dreapta?

- Monad m => a -> [a] -> m [a] ✓
- Monad m => a -> [m a] -> [m a]
- a -> [a] -> Monad [a]
- nicio variantă

Correct

Response saved

```
\x xs -> return (x : xs)
```

Ce tip are expresia din dreapta?

- Monad m => a -> [a] -> m [a]
- Monad m => a -> [m a] -> [m a]
- a -> [a] -> Monad [a]
- nicio variantă ✓

Correct

Response saved

```
(\x xs -> return x) : xs
```

1. Ce tip are o funcție care primește ca parametru o listă de șiruri de caractere și calculează maximul lungimilor șirurilor?

nu se poate defini o astfel de funcție

f :: [String] -> Int ✓

f :: [Char] -> Int

f :: String -> Int

Correct

Response saved

2. Ce eroare este în codul din dreapta?

- codul este corect ✗
- funcția h nu este apelată corect
- indentarea este greșită
- construcția let nu este folosită corect ✓

Incorrect

Response saved

```
g x = x + h x
    let h x = x + 1
```

3. Care din următoarele instrucțiuni este o funcție anonimă?

f y = y * 2

\x y -> (x + y)^2

\y . y ^ 1

nicio variantă

Correct

Response saved

4. Care dintre definițiile de mai jos este corectă?

data Tree a = Empty | Branch a (Tree a) (Tree a)

data Tree = Empty | Leaf a | Branch (Tree a) (Tree a)

data Tree a = Empty | Leaf a | Branch (Tree) (Tree)

data Tree = Leaf | Node a Tree Tree

Incorrect

Response saved

5. Ce calculează funcția f de mai jos?

definiție incorectă

funcția va întoarce True mereu

dacă toate șirurile din lista au cel mult lungime 4

niciun răspuns de mai sus

Correct

Response saved

```
f xs = foldr (||) True  [0 <= length x && length x < 5 | x <- xs]
```

6. Ce tip are expresia de mai jos?

[Char] -> [(Char,Char)]

[(Char,[Char])]

instrucțiune invalidă ✓

[[(Char,[Char])]] -> [[(Char,[Char])]]

Correct

Response saved

```
filter (\(x,y) -> x > y) [('a',"a"),('b',"b"),('c',"d")]
```

7. Cum puteți obține valoarea (3,'y') folosind listele din dreapta?

- l3
- nicio varianta ✓
- l3 !! 1
- l3[1]

Correct

Response saved

```
l1 = ['z','y'..]
l2 = [1,3..]
l3 = take 4 $ zip l1 l2
```

8. Care operație de mai jos produce rezultatul [5,6,7,8]?

filter id [5,6,7,8]

fold id [5,6,7,8]

map id [5,6,7,8]

nicio variantă

Correct

Response saved

9. Care din implementările de mai jos este corectă pentru funcția hof de mai jos?

- nicio variantă
- hof g h e = g e (h e) ✓
- hof g h e = g h e
- hof g h e = g (h e)

Correct

Response saved

```
hof :: (a -> b -> c) -> (a -> b) -> a -> c
```

10. În tipul algebric de date din dreapta, care este constructorul de tip?

Type, Case1

Case1, Case2, Case3

nicio variantă ✓

Type, Case1, Case2, Case3

Correct

Response saved

```
data Type = Case1
          | Case2
          | Case3
```